SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Vulnerabilities and security controls across mobile, web, and cross-domain application development

Presented by

Mr. Abhijeet Pasi

Department of Computer Engineering

K J Somaiya School of Engineering

Outline

- Introduction
- Mobile Application Vulnerabilities and Controls
- OWASP Mobile Vulnerabilities
- Top Mobile Vulnerabilities
- Mobile Application Controls

Introduction

- Why do applications across different domains face unique security challenges?
- A mobile app running on a user's phone, a web application accessed through browsers, and a system that allows cross-domain data sharing all operate in fundamentally different environments.
- Each environment introduces distinct vulnerabilities and requires tailored controls.

Learning Objectives

By the end of this session, you will be able to:

- Identify the top vulnerabilities in mobile, web, and cross-domain applications
- Explain the specific controls required for each development domain
- Apply security best practices to real-world development scenarios

Mobile Application Vulnerabilities and Controls

The Mobile Threat Landscape

Mobile applications face unique risks because they operate in an untrusted environment. Unlike web applications that run on servers you control, mobile apps run on user devices that may be:

- Jailbroken or rooted
- Connected to insecure networks
- Shared between multiple users
- Running malicious background processes

Nearly 70% of organizations are not currently obfuscating their mobile code, leaving their applications vulnerable to static analysis attacks

OWASP Mobile Vulnerabilities

Rank	Vulnerability	Description
M1	Improper Platform Usage	Misuse of platform features like Android intents, TouchID, or Keychain
M2	Insecure Data Storage	Storing sensitive data where malware or other apps can access it
M3	Insecure Communication	Weak TLS, improper certificate validation, cleartext transmission
M7	Insufficient Binary Protections	No obfuscation, easy reverse engineering, code tampering

Top Mobile Vulnerabilities

Insecure Data Storage (M2)

Mobile developers often assume users won't access the device filesystem. This leads to storing:

- Passwords in plain text local databases
- API keys in shared preferences
- Sensitive data in unencrypted logs

Example Scenario: A banking app stores user transaction history in an unencrypted SQLite database. A malicious app with storage permissions can read this data.

Top Mobile Vulnerabilities

Insufficient Binary Protections (M7)

Without code obfuscation, attackers can:

- Reverse engineer the app to extract hardcoded secrets
- Repackage the app with malware
- Bypass license checks

Insecure Communication (M3)

Mobile apps often:

- Accept any SSL certificate (no validation)
- Fail to enforce TLS
- Transmit sensitive data over HTTP on untrusted Wi-Fi

Mobile Application Controls

Control 1: Secure Data Storage

- Use platform-specific encrypted storage: Android's EncryptedSharedPreferences, iOS Keychain
- Avoid storing sensitive data in logs, backups, or world-readable files
- Encrypt data at rest using AES-256

Control 2: Code Hardening and Obfuscation. Implement multi-layered code hardening, including :

- Name obfuscation
- Control flow obfuscation
- Code virtualization and Data encryption

Mobile Application Controls

Control 3: Runtime Protection (RASP)

Runtime Application Self-Protection checks monitor app behavior and provide automated defenses:

- Anti-debugging capabilities
- Detection of tampered system libraries
- Blocking emulators and virtual environments
- Preventing function hooking

Mobile Application Controls

Control 4: Secure Communication

- Enforce TLS 1.2 or higher
- Implement certificate pinning to prevent MITM attacks
- Never accept all certificates

Control 5: App Attestation

- Verify that the mobile app interacting with your APIs is authentic, untampered, and running on a secure device

Web Application Vulnerabilities and Controls

The Web Application Threat Landscape

Web applications are inherently public-facing, making them prime targets for attackers.

Nearly 35% of organizations report experiencing a "significant security incident" at least once a week

OWASP Web Top Vulnerabilities

1. Broken Access Control

When applications fail to enforce restrictions on authenticated users, attackers can access unauthorized data or functionality .

Example: A user with standard privileges modifies the URL from /user/dashboard to /admin/panel and gains administrative access.

2. Injection Flaws

SQL injection, NoSQL injection, and command injection occur when untrusted data is sent to an interpreter as part of a query or command .

Example: An attacker enters ' OR '1'='1 in a login form, bypassing authentication.

3. Cross-Site Scripting (XSS)

Attackers inject malicious scripts into web pages viewed by other users, stealing session tokens or personal information .

4. Security Misconfigurations

OWASP Web Top Vulnerabilities

3. Cross-Site Scripting (XSS)

Attackers inject malicious scripts into web pages viewed by other users, stealing session tokens or personal information .

4. Security Misconfigurations

Includes default credentials, overly detailed error messages, unsecured cloud storage, and unnecessary enabled features .

5. Vulnerable and Outdated Components

Third-party libraries and frameworks with known vulnerabilities create entry points for attackers

Web Application Controls

Control 1: Input Validation and Sanitization

Modern input validation requires a **contextual approach** where validation matches the specific data sink :

- Identify all data entry points (parameters, headers, cookies)
- Apply different validation rules based on whether data flows into SQL queries, HTML output, or JavaScript
- Use allowlisting rather than blocklisting

Web Application Controls

Control 2: Secure Authentication and Session Management

- Implement Multi-Factor Authentication (MFA)
- Use strong password policies
- Secure session tokens with proper expiration
- Implement OAuth 2.0 and OpenID Connect where appropriate

Control 3: Content Security Policy (CSP)

CSP dramatically reduces XSS impact by controlling what resources browsers can load

Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'

Deploy initially in Report-Only mode to identify issues before enforcement

Web Application Controls

Control 4: Web Application Firewalls (WAF)
WAFs identify and filter malicious requests like injection and XSS attacks, reducing automated threats

Control 5: Secure Configuration and Hardening

Minimize attack surface by disabling unnecessary features and plugins

Harden server and database access with least privilege principles

Implement security headers: X-Frame-Options, HSTS

Control 6: Dependency Management

Regularly scan third-party libraries for vulnerabilities using tools like OWASP Dependency-Check

Web Application Controls

Control 7: API Security

- Implement rate limiting and throttling
- Validate all inputs
- Encrypt all communications with TLS
- Follow OWASP API Security Top 10 guidelines

Cross-Domain Application Vulnerabilities and Controls

Understanding Cross-Domain Applications

- Cross-domain applications allow resources from different origins to interact.
- This is essential for modern web functionality but introduces significant security risks.

The Same-Origin Policy

- Browsers enforce the **Same-Origin Policy (SOP)** , which permits scripts from one web page to access data from another page only if they share the same protocol, host, and port.
- Cross-domain mechanisms intentionally bypass this restriction, creating risk.

How Cross-Domain Attacks Work

Flash and Silverlight Policy Files

Cross-domain policy files (crossdomain.xml for Flash, clientaccesspolicy.xml for Silverlight) list domains allowed to make cross-domain requests. If these files are overly permissive (using wildcards or including untrusted domains), attackers can :

- Transfer private information including cookies and session data
- Send malicious requests to websites on behalf of victims
- Launch attacks without victim awareness

CORS Misconfigurations

CORS (Cross-Origin Resource Sharing) headers control cross-origin requests made by browsers. A common vulnerability is dynamically setting Access-Control-Allow-Origin based on user-supplied Origin headers without validation

Cross-Domain Controls

Control 1: Strict Origin Validation

- Never trust user-supplied Origin headers. Implement a **whitelist of trusted domains**

Control 2: Restrict Cross-Domain Policy Files

For Flash and Silverlight policy files :

- Avoid using wildcards (*) in cross-domain policy files
- Explicitly list only trusted domains
- Regularly audit policy files for misconfigurations

Control 3: Implement Content Security Policy (CSP)

- CSP headers limit where resources can be loaded from, providing defense-in-depth against cross-domain attacks

Cross-Domain Controls

Control 3: Implement Content Security Policy (CSP)

- CSP headers limit where resources can be loaded from, providing defense-in-depth against cross-domain attacks

Control 4: Monitor for Unusual Cross-Origin Requests

- Review server logs for unexpected Origin headers
- Set up alerts for suspicious referrers
- Monitor for patterns indicating cross-origin probing

Control 5: Network Segmentation

Isolate applications handling sensitive data within segmented network zones to limit lateral movement if cross-domain vulnerabilities are exploited

4.2 Secure Coding Practices, Cryptographic Libraries and Tools for Secure Coding

Introduction

The Cost of Insecure Code

- Building secure software from the beginning is significantly cheaper than fixing security issues after deployment .
- Security breaches carry substantial costs—financial, reputational, and regulatory.
- According to SANS 2009 study, more than 60% of attacks on the internet target web applications at the application layer .

Core Security Goals

- The goal of software security is to maintain the confidentiality, integrity, and availability (CIA triad) of information resources .
- Every secure coding practice ultimately serves one or more of these principles.

Core Secure Coding Practices

Input Validation

Input validation is the first line of defense. All untrusted data must be validated before processing

Critical Principles:

- Perform data validation on the server — never trust client-side validation alone
- Classify data sources as trusted or untrusted; validate all untrusted sources
- Use centralized input validation routines for consistency across the application
- Validate all input before processing: parameters, headers, cookies, and values

Core Secure Coding Practices

Input Validation

- Input validation is the first line of defense.
- All untrusted data must be validated before processing

Critical Principles:

- Perform data validation on the server — never trust client-side validation alone
- Classify data sources as trusted or untrusted; validate all untrusted sources
- Use centralized input validation routines for consistency across the application
- Validate all input before processing: parameters, headers, cookies, and values

Core Secure Coding Practices

What to Validate:

- Expected data types
- Data range and length
- Character sets (specify UTF-8 for all sources)

Allowlisting vs. Blocklisting:

- Validate all input against an allowlist (whitelist) of allowed characters whenever possible.
- If hazardous characters must be allowed, implement additional controls like output encoding

Core Secure Coding Practices

Output Encoding

- Encoding prevents injection attacks by ensuring that untrusted data is treated as data, not as code

Key Practices:

- Perform all encoding on a trusted system (the server)
- Utilize a standard method for each type of output encoding
- Encode output based on the context it will be in HTML, JavaScript, SQL, or XML each require different encoding
- Encode all characters unless they are known to be safe for the intended interpreter

Authentication and Password Management

Secure authentication is critical.

Password Storage:

- Use **cryptographically strong one-way salted hashes** for passwords
- **Do not use MD5** — it is cryptographically broken
- Password tables and keys should be writeable only by the application
- Password hashing must be implemented on a trusted system (server)

Authentication Controls:

- Require authentication for all routes and resources except those intentionally public
- Use standard, tested authentication services rather than custom implementations
- Implement centralized authentication logic
- All authentication controls should **fail securely**

Authentication and Password Management

Password Policies:

- Enforce password complexity and length requirements
- Implement account lockout after invalid attempts (temporary lock, not so long as to enable DoS)
- Prevent password reuse
- Notify users when password reset occurs

Error Messages:

- Authentication failures should **not reveal which part was incorrect**
- Use generic messages like "Invalid username and/or password" rather than "Invalid username" or "Invalid password"
- Ensure errors are identical in both display and source code

Authentication and Password Management

Session Management

Session identifiers must be protected to prevent hijacking .

Critical Practices:

1. **Use the server or framework's session management controls** — don't roll your own
2. Session identifiers must be created on a trusted system using well-vetted algorithms ensuring sufficient randomness
3. **Do not expose session IDs in URLs, error messages, or logs** — they should only be in HTTP Cookie headers
4. Implement session inactivity timeout — in most cases, no more than several hours
5. Generate a new session ID on re-authentication and when switching from HTTP to HTTPS
6. Set the secure attribute for cookies transmitted over HTTPS
7. Set cookies with the HttpOnly attribute unless client-side access is specifically required

Authentication and Password Management

Access Control

Access control ensures users can only access resources they are authorized to use .

Essential Principles:

- Use only trusted system objects (server-side session objects) for access decisions
- Use a single site-wide component to check access authorization
- Access controls should **fail securely** — deny all access if configuration cannot be accessed
- Enforce authorization controls on **every request**, including AJAX and Flash requests
- Restrict access to protected URLs, functions, services, and application data
- **Separate privileged logic** from other application code

What is Standardized Testing Policy?

Definition: A formal set of rules governing the administration, scoring, and use of standardized tests.

Key Policy Pillars:

- **Fairness:** All test-takers receive the same conditions.
- **Validity:** Scores accurately reflect ability, not external factors.
- **Reliability:** Consistent results across different administrations.
- **Accountability:** Clear consequences for policy violations.

Core Elements of Testing Policy

Element	Description
Test Administration Protocol	Who, when, where, how long
Accommodations	For ELL, IEP, 504 plans (without altering construct)
Data Privacy	FERPA, GDPR, or local laws on student data
Breach Procedures	What constitutes a breach? What are the penalties?
Retest & Score Verification	Conditions for invalidating or rescoring

Why Security Requirements Matter

- **Prevents:** Cheating, prior knowledge, item exposure, and impersonation.
- **Protects:** Test integrity, credential value, fair competition (e.g., college admissions).
- **Mitigates:** Legal liability, media scandals, loss of funding/recognition.

Security Requirements – Physical Environment

- **Secure storage:** Locked, access-controlled cabinets/rooms for test booklets.
- **Chain of custody:** Sign-in/sign-out logs for every test booklet or device.
- **Environment:** No unauthorized posters, notes, or electronic devices.
- **Seating:** Spaced desks, partitions if needed, clear line of sight.

Security Requirements – Digital & Personnel

Digital:

- Lockdown browsers (disable copy/paste, print, other apps).
- Randomized question pools.
- Proctoring software (AI or live).

Personnel:

- Background checks for proctors.
- Mandatory training & certification.
- Non-disclosure agreements (NDAs).

Security vs. Testing Security

Aspect	General Security	Testing Security
Focus	People, facilities, data	Test content & scores
Threats	Theft, hacking, vandalism	Cheating, pre-knowledge, leakage
Controls	Locks, firewalls, guards	Proctors, timing, form randomization
Example	Prevent server breach	Prevent student looking at another's screen

Secure Testing Approach – The Three Phases

- **Pre-Test Security** (Planning & Setup)
- **During-Test Security** (Active Monitoring)
- **Post-Test Security** (Collection & Analysis)

Phase 1 – Pre-Test Security

- **Inventory control:** Count and log all materials before distribution.
- **Train proctors:** Scripted instructions, breach scenarios.
- **Authenticate test-takers:** Photo IDs, biometrics, or secure login tokens.
- **Secure the room:** Remove smart watches, bags, notes.
- **Randomisation:** Assign different test forms or item orders.

Secure Testing Approach – The Three Phases

Phase 2 – During-Test Security (Active)

- **Proctoring ratios:** Typically 1:30 (in person), 1:15 for high-stakes.
- **Live monitoring:** Walk around, no reading/sitting idle.
- **Digital proctoring:** Flags for eye movement, background noise, multiple faces.
- **Incident logs:** Record any disruption, irregularity, or suspected cheating.
- **Timing enforcement:** Strict start/stop; no extra time unless documented.

Phase 3 – Post-Test Security

- **Immediate collection:** Count all materials again; match to pre-test count.
- **Secure transmission:** Encrypted uploads for digital; courier for paper.
- **Data forensics:** Post-administration statistical analysis (e.g., erasure analysis, answer copying indices).
- **Retention policy:** Destroy or archive securely per policy (e.g., 1 year for paper, encrypted for digital).

Common Security Breaches & Mitigations

Breach	Mitigation
Student shares answers	Multiple equivalent forms
Proctor gives hints	Scripted, recorded instructions
Test booklet stolen	Daily inventory, locked storage
Hacked testing platform	Two-factor auth, IP restriction
Cell phone use	Bags in lockers / signal blockers

Case Study

Atlanta Public Schools Cheating Scandal (2014)

- **Issue:** Educators changed student answers after testing.
- **Lack of:** Post-test erasure analysis, chain of custody for answer sheets.
- **Outcome:** 11 convicted of racketeering; tests invalidated.
- **Lesson:** Security must include *post-test forensic checks*.

Building a Secure Testing Culture

Not just rules – mindset.

- **Training:** Annual refreshers with real breach examples.
- **Reporting:** Anonymous hotline for integrity concerns.
- **Proportional consequences:** Students & staff know penalties.
- **Transparency:** Explain *why* security protects them.

Summary Checklist for Testing Coordinators

- Policy reviewed & posted
- All materials inventoried
- Proctors certified & assigned
- Room cleared of prohibited items
- Authentication method ready
- Breach response plan known
- Post-test collection & analysis scheduled

Group Discussion Question

Discuss as a group: Policy, ethics, and operational steps.